Overview by Abel Chuchu

Acoustic Drone Detection Project

This project builds a machine learning system that detects whether a drone is present in an audio recording.

Instead of using cameras or radar, the system listens to sound and tries to recognize the acoustic pattern of drones.

The project follows a complete machine learning pipeline:

```
Audio Dataset
     ↓
Audio Visualization & Exploration
     ↓
Audio Preprocessing
     ↓
Feature Extraction (Mel-Spectrogram)
     ↓
CNN Model Training
     ↓
Model Evaluation
     ↓
Prediction System
     ↓
Interactive Detection Dashboard
     ↓
Simulation of military drone sounds
```

1. Dataset Used

The model was trained on a dataset called DADS (Drone Audio Detection Samples).

Dataset properties:

- Large public drone audio dataset
- Audio format: WAV
- Sampling rate: 16 kHz
- Two classes:

```
0 → No drone (background noise)
1 → Drone
```

Examples of sounds in the dataset:

Drone sounds:

- quadcopter drones
- electric propeller noise

Non-drone sounds:

- city noise
- wind
- environment sounds

2. Using a Dataset Subset

The full dataset is about 6.6 GB, which is too large for quick experiments. So the project randomly selects 6,000 audio samples This subset is used for training and testing the model. This keeps:

- training fast
- memory usage manageable

3. Audio Exploration

Before training the model, the notebook first visualizes the audio data. This helps understand how drone sound differs from normal noise. Three main visualizations

are used.

3.1 Waveform

A waveform shows Amplitude vs Time

Drone audio tends to have:

- repeating patterns
- steady propeller vibration

Background noise is usually:

- irregular
- random

3.2 Mel-Spectrogram

This is the **most important representation** in the project. The audio is converted into an image. A Mel-Spectrogram shows:

X axis → Time
Y axis → Frequency
Color → Sound intensity

Example patterns:

Drone sound- strong horizontal bands (propeller frequencies)

Noise- more random frequency distribution

This spectrogram becomes the input to the neural network.

3.3 MFCC Features

MFCC stands for Mel Frequency Cepstral Coefficients. These capture the texture of sound and act like a fingerprint for audio signals. They help show acoustic differences between classes.

4. Preprocessing Pipeline

Before training the model, every audio file must be converted into a standard format. The preprocessing pipeline performs several steps.

```
Raw audio
     ↓
Trim or pad audio to fixed length
     ↓
Generate Mel-Spectrogram
     ↓
Convert to RGB image
     ↓
Resize to 128×128
     ↓
Normalize values
     ↓
Convert to tensor (PyTorch format)
```

So every sample becomes 128 × 128 spectrogram image

This image is what the neural network learns from.

5. Dataset Splitting

The dataset is divided into three groups.

```
70% → Training set
15% → Validation set
15% → Test set
```

Purpose of each:

Training set → teaches the model

Validation set → monitors performance during training

Test set → final evaluation on unseen data

6. CNN Model

The model used is a Convolutional Neural Network (CNN). CNNs are commonly used for image recognition and pattern detection. Since spectrograms are images, CNNs work very well.

Model Structure

The architecture contains:

Feature Extraction Layers

These layers detect patterns.

```
Input Spectrogram
      ↓
Convolution Layer
      ↓
ReLU Activation
      ↓
Max Pooling
```

This process is repeated several times.

Each layer learns edges, shapes, patterns

Classification Layers

After features are extracted:

```
Flatten features
      ↓
Fully connected layer
      ↓
Output layer
```

The output produces two probabilities:

```
P(no drone)
P(drone)
```

The highest probability becomes the prediction.

7. Model Training

The model is trained using **supervised learning**. This means the correct labels are known during training. Each training step:

```
Input spectrogram
     ↓
Model predicts class
     ↓
Compare prediction with true label
     ↓
Calculate loss (error)
     ↓
Update model weights
```

The goal is to minimize prediction error.

Training Details

Training parameters:

Epochs: 20
Optimizer: Adam
Loss function: CrossEntropyLoss

Additional techniques used:

Learning Rate Scheduler

Automatically reduces learning rate if improvement stops.

Early Stopping

Stops training if validation loss does not improve for 5 epochs.

This prevents overfitting.

8. Model Evaluation

After training, the model is tested on the **test dataset**.

Several metrics are calculated.

Accuracy - Percentage of correct predictions.

Confusion Matrix- Shows prediction performance.

- False negatives are the most dangerous errors in real systems.

ROC Curve- Shows how well the model separates the two classes.

The AUC score summarizes performance.

9. Single File Prediction

The notebook creates a function called predict(audio)

This function:

```
Audio input
     ↓
Preprocess
     ↓
Spectrogram
     ↓
CNN model
     ↓
Prediction
```

Example output:

```
Drone detected
Confidence: 94%
```